

Transductive versus Inductive Link Prediction on Rule-Generated Family Trees: TransE, Entity Fold-in and 2-Hop Relational Message Passing

Plover-64D

STEM seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/rule-structured-synthetic-knowledge-graph-trans>

Abstract—We ask how much of a knowledge-graph completion model’s accuracy survives a move from held-out triples of its training graph to a freshly generated graph with entirely new entities, in a setting where the relations are derived by rules and the answer structure is known by construction. We generate family-tree graphs of 500 people with four relations (parent, sibling, grandparent, uncle/aunt), and compare a tuned, reimplemented TransE, TransE with entity fold-in, a 2-layer query-conditioned relational message-passing model (Path-MP, a reduced NBFNet-style network) and a hand-written one-step rule oracle, over 5 seeds that each use a different graph pair. Path-MP reaches filtered MRR 0.988 on held-out triples and 0.989 on the new graph, with no measurable drop; TransE reaches 0.658 and falls to 0.011. Fitting new entity vectors with frozen relation vectors restores TransE to 0.644, still far below Path-MP. Ablations show that one message-passing layer is at chance, a third layer adds nothing measurable, training without removing the query edge costs about half the MRR, and when only a fraction of the new graph is observed Path-MP degrades quickly (MRR 0.292 at a quarter of the triples) and is no longer distinguishable from fold-in TransE. The study is small, synthetic and has a high ceiling for the path model; it measures a gap that is known in principle, and we report where it is and is not there.

Index Terms—knowledge graph completion, inductive link prediction, TransE, relational message passing, synthetic benchmark, rule-derived relations

I. Introduction

Knowledge-graph completion models are usually evaluated by hiding a fraction of the triples of one graph and asking the model to rank the missing answers. Embedding models such as TransE [1], DistMult [2], ComplEx [3], RotatE [4] and ConvE [5] assign one free vector to every training entity and are therefore transductive: an entity that appears only after training has no learned representation. When tuned carefully they are strong on this protocol [6]. Subgraph and path-based networks, in contrast, compute a representation of a candidate answer from the relations on the paths that reach it from the

query entity [7]–[10]. They have no per-entity parameters, so they can be applied to a graph with new entities, and the same is true for rule learners such as Neural-LP and DRUM [11], [12]. The inductive setting has been studied on splits derived from existing benchmarks [7], [13], and recent work trains one model to transfer to unseen graphs [14].

On those benchmarks the rules that generate the test triples are unknown, so it is hard to say why a model succeeds or fails and what the ceiling is. We therefore use a deliberately simple world where everything is known: a family tree with parent facts and relations derived from them by Horn rules (sibling, grandparent, uncle/aunt). We ask three questions. (i) On held-out triples of the training graph, does a 2-hop path model beat TransE, and by how much? (ii) When the test graph is a freshly generated one with new entities, how large is the loss for each model, and is it relation-specific? (iii) Is the embedding baseline’s failure on new entities a property of the model, or of how the comparison is set up, i.e. does a simple fold-in of the new entities’ vectors recover most of it? We add ablations on depth, on the removal of the query edge during training, and on the density of the observed part of the new graph.

Contributions. (1) A small, fully reproducible generator of rule-derived family-tree knowledge graphs with a split that cannot leak through the symmetry of the sibling relation. (2) A paired comparison, over 5 seeds, each with its own pair of graphs, of tuned TransE, TransE with fold-in, a 2-layer relational message-passing model and a rule oracle, in both settings. (3) Ablations showing which parts of the path model matter and when its advantage over fold-in TransE disappears. This is a controlled CPU study on a synthetic world, not a new method and not a benchmark result; Section VII lists what it does not show.

II. Related Work

Embedding models. TransE [1] models a relation as a translation in embedding space; DistMult [2], ComplEx [3] (complex bilinear scoring), RotatE [4] (rotations, able to represent symmetric and compositional patterns) and

Plover-64D is the codename of an autonomous study, not a person. Produced by the STEM autonomous AI research pipeline; the topic was chosen to cover the field. Every reported result is traced to logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

ConvE [5] improve on it. A large re-evaluation found that classic models are competitive once their hyperparameters and training are tuned [6], which motivates our tuning of the TransE baseline. PyKEEN [15] provides such baselines and small rule-rich datasets; we reimplement TransE instead, to control the splits, the fold-in procedure and the tie policy.

Rules and paths. Neural-LP [11] and DRUM [12] learn soft chains of relations and are inductive. GraIL [7] classifies the subgraph enclosing a candidate triple and introduced inductive splits in which training and test graphs share no entities. NBFNet [8] generalises path-based reasoning to a Bellman-Ford style message-passing network conditioned on the query; our Path-MP is a reduced version of this idea (fixed depth, sum aggregation, no learned path pruning). Relational message passing [9] and its fully inductive variant [10] pass messages over relations rather than entities. R-GCN [16] mixes message passing with entity-specific parameters and is thus still transductive. ULTRA [14] pre-trains one model that transfers to unseen graphs and relation vocabularies. Open problems for the inductive setting, including benchmark design, are discussed by Galkin et al. [13].

What is missing. The inductive benchmarks above are derived from existing graphs whose generative rules are unknown; the mix of rule-inferable and non-inferable test triples is unknown as well. Our contribution is the opposite trade: a toy world in which each relation is exactly rule-derived, so that differences between models, and between the two settings, can be attributed to model properties and observed-graph density rather than to unknown benchmark structure. This comes at the cost of realism, which we discuss in Section VII.

III. Method

A. Data: rule-derived family graphs

A graph has $n = 500$ people and four base relations $\mathcal{R} = \{\text{parent, sibling, grandparent, uncle}\}$. Parent facts come from a random multi-generation family forest (Section IV). The other relations are derived by the rules

$$\text{sibling}(x, y) \leftarrow \text{parent}(p, x) \wedge \text{parent}(p, y) \wedge x \neq y, \quad (1)$$

$$\text{grandparent}(x, y) \leftarrow \text{parent}(x, z) \wedge \text{parent}(z, y), \quad (2)$$

$$\text{uncle}(x, y) \leftarrow \text{sibling}(x, z) \wedge \text{parent}(z, y), \quad (3)$$

where uncle is gender-neutral (a sibling of a parent: an uncle or an aunt), so that every relation follows from the parent facts alone. Every triple (h, r, t) also induces an inverse edge (t, r^{-1}, h) , giving $2|\mathcal{R}| = 8$ edge types. Tail prediction for $(h, r, ?)$ and head prediction for $(?, r, t)$ (asked as $(t, r^{-1}, ?)$) are both evaluated.

B. TransE (reimplemented)

Each entity e and relation r has a vector $\mathbf{e}, \mathbf{r} \in \mathbb{R}^d$. The score of a triple is $f(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_1$ (lower is better). Training minimises the margin ranking loss

$$\mathcal{L} = \frac{1}{|B|k} \sum_{(h,r,t) \in B} \sum_{j=1}^k \max(0, \gamma + f(h, r, t) - f(h'_j, r, t'_j)), \quad (4)$$

where (h'_j, r, t'_j) replaces the head or the tail (with probability $\frac{1}{2}$ each) by a uniformly random entity, and entity vectors are projected to $\|\mathbf{e}\|_2 \leq 1$ after every step. Ranking a head query uses $f(c, r, t)$ over all candidates c . A new entity has no trained vector, so in the inductive setting TransE (frozen) gives every new entity a fresh random vector drawn from the initialisation law; the relation vectors are those learned on graph 1.

C. TransE with fold-in (reimplemented)

To give the embedding baseline a fair chance on new entities, we keep the relation vectors fixed and fit only the new entity table on the observed triples of the new graph, with the same loss and optimiser. No held-out test triple is used. This is a standard way to embed out-of-sample entities but needs gradient steps on the new graph at test time.

D. Path-MP: 2-hop relational message passing

Path-MP scores every entity v as an answer to a query $(h, q, ?)$ with q one of the 8 edge types. Let $\mathbf{q} \in \mathbb{R}^d$ be a learned embedding of the query relation and $\mathbf{x}_v^0 = \mathbb{I}[v = h] \mathbf{q}$. For layers $\ell = 1, \dots, L$,

$$\mathbf{w}_e^\ell = \text{reshape}(W^\ell \mathbf{q})_e \in \mathbb{R}^d, \quad (5)$$

$$\mathbf{a}_v^\ell = \sum_{(u,e,v) \in \mathcal{E}} \mathbf{x}_u^{\ell-1} \odot \mathbf{w}_e^\ell, \quad (6)$$

$$\mathbf{x}_v^\ell = \text{ReLU}(\text{LN}(U^\ell[\mathbf{x}_v^{\ell-1}; \mathbf{a}_v^\ell]) + \mathbf{x}_v^0), \quad (7)$$

where $\mathcal{E}$ is the set of (forward and inverse) edges of the message graph, e indexes edge types, $\odot$ is the element-wise product, $W^\ell \in \mathbb{R}^{8d \times d}$ makes the per-edge-type weights depend on the query relation, U^ℓ is linear, LN is layer normalisation, and the initial state $\mathbf{x}^0$ is added at every layer as a boundary condition. The score is $s(v) = \text{MLP}(\mathbf{x}_v^L)$. Training uses 1-vs-all cross-entropy over all n entities, $\mathcal{L} = -\log \text{softmax}(s)_t$, where other training-known answers of the same query are masked out. Both directions of every training triple are used as queries. Crucially, while training on query (h, r, t) the edge (h, r, t) and its inverse (and, for the symmetric sibling relation, the mirror triple) are removed from the message graph of that query; otherwise the model could read the answer off the graph. At test time the message graph is the full observed graph of the setting. The model has no entity-specific parameters, hence the same weights apply to any graph. Depth L is 2 in the main experiments: each rule above has a body of length two, so two hops suffice to reach the answer.

E. Rule oracle (hand-written)

As a reference we score each candidate by the number of groundings of a one-step rule set applied to the observed graph (with A_r the adjacency matrix of relation r): parent $\leftarrow A_p A_s$ (a parent of my sibling is my parent); sibling $\leftarrow (A_p^\top A_p)$ with zeroed diagonal, plus symmetry, transitivity and $A_u A_p^\top$; grandparent $\leftarrow A_p A_p + A_p A_u + A_g A_s$; uncle $\leftarrow A_s A_p + A_u A_s$, where subscripts p, s, g, u abbreviate the four relations. It is not trained, and it is not a ceiling: it applies each rule once to observed facts, while the learned model may combine several rule paths.

F. Evaluation

For each held-out triple we rank the true answer among all n entities after filtering every other entity that forms a true triple (in the whole generated graph) with the query. A tie between the true answer and other candidates is resolved by counting half of the tied candidates, rank = $1 + |\{s > s_t\}| + \frac{1}{2}|\{s = s_t\}|$, so constant scores cannot score well. We report MRR over head and tail queries, Hits@1, Hits@10 and the MRR per relation.

IV. Experimental Setup

Data generation. A graph is grown from 6 founding couples. Each couple has between one and four children (probabilities 20, 35, 30 and 15 percent for one to four children), each child marries a newly created outsider with probability 80 percent, and the new couple is queued, up to 6 generations of couples, until exactly 500 people exist. Entity ids are randomly permuted so that ids carry no information. A graph has about 2028 triples (mean over seeds) spread almost evenly over the four relations (relation shares between 22.5% and 27.2% on average).

Splits and the two settings. Triples are split at the level of units (80/10/10 percent for train, validation and test); both directions of a sibling pair form one unit, so a held-out sibling triple has no mirror in the training data. Models are trained on the train+validation units of graph 1. Transductive test: the held-out units of graph 1 (about 206 triples per seed), message graph = train+validation units of graph 1. Inductive test: a second graph with 500 entities that were never seen, generated independently, split in the same proportions; message graph and fold-in data = its train+validation units; test = its held-out units (about 203 triples per seed). Both settings are therefore evaluated with the same fraction of the graph observed and the same number of test triples. A seed fixes both graphs, both splits, the initialisation and the minibatch order, so each seed is an independent graph pair and comparisons are paired by seed.

Systems and hyperparameters. All learning systems are reimplemented in PyTorch (CPU, 2 threads) in one script (method/run.py) with one evaluation function. TransE: dimension 64, L_1 distance, margin $\gamma = 8$, $k = 10$ negatives, Adam with learning rate 0.01, batch size 512, 200 epochs;

fold-in uses the same optimiser for 200 epochs on the new graph. Path-MP: width $d = 32$, $L = 2$, Adam with learning rate 0.003, batches of 32 queries, 3 epochs, gradient-norm clipping at 5.

Tuning. Hyperparameters were chosen on the validation units of graph 1 for two separate graph seeds (2 seeds distinct from the evaluation seeds); the test units were never used. TransE received three rounds of search (margin in $\{1, 2, 4, 6, 8, 12, 16\}$, L_1 versus L_2 distance, dimension 64 versus 128, 200 versus 400 epochs), because the best margin was at the edge of the first grid; the optimum (margin 8, L_1 , dimension 64, 200 epochs) lies inside the grid, and larger margins, the larger dimension (tried at margin 4 only) and more epochs (tried at margins 4, 8 and 12) did not improve validation MRR (all 28 tuning runs are logged in group tune_transe). Path-MP received a 2×2 grid (epochs 3 or 6, learning rate 0.003 or 0.01); 3 and 6 epochs were tied on validation and the cheaper 3 epochs were used. Path-MP therefore received a smaller tuning budget than TransE, which, if anything, favours the baseline (Section VII).

Statistics and runs. 5 seeds (0–4). Tables report mean $\pm$ standard deviation over seeds; tests are paired t -tests over seeds from rh compare or from the analysis script (method/analyze.py, logged as a run). With five seeds these tests have little power for small effects and we do not interpret non-significant results as equivalence. Every run was executed with the rh run wrapper on a shared CPU machine; each run takes well below the per-run limit. The registry also contains failed rows that we did not hide: 30 launches of the sweep_density baselines (TransE+fold-in and the rule oracle) were corrupted by a variable clash in the launcher script and were re-run with the corrected script (the Path-MP rows of that sweep were unaffected), and one analysis-script launch failed on a registry control row before the script was fixed.

V. Results

Tables I and II give the main results; Fig. 1 plots MRR per system and setting. All numbers are means over 5 seeds; differences quoted below are paired over seeds.

H1: Path-MP beats TransE on held-out triples (supported). Path-MP reaches MRR 0.988 against 0.658 for tuned TransE, a paired difference of 0.330 (paired $p = 7.98 \times 10^{-6}$ over 5 seeds, the test registered in the proposal). The picture is different for Hits@10: TransE reaches 0.983 against 0.990, but Hits@1 is 0.424 against 0.985. TransE places the answer among the first ten candidates almost always but often not first, which, under a 500-entity candidate set, is a weak criterion; MRR and Hits@1 expose the difference. Path-MP also exceeds the one-step rule oracle (MRR 0.937, difference 0.051, paired $p = 9.61 \times 10^{-3}$). The oracle is a one-step reference, not a ceiling: the learned model can combine several rule paths,

TABLE I

Main results: filtered MRR, Hits@1 and Hits@10 (mean $\pm$ std over seeds) on held-out triples of the training graph (transductive) and on a fresh graph with new entities (inductive). Bold: Path-MP; underline: best baseline per column and setting.

Method	Task	mrr $\uparrow$	hits1 $\uparrow$	hits10 $\uparrow$	n
Rule oracle (hand-written)	transductive	0.937 $\pm$ 0.026	0.931 $\pm$ 0.026	0.940 $\pm$ 0.026	5
TransE+fold-in (reimplemented)	transductive	0.656 $\pm$ 0.029	0.423 $\pm$ 0.056	0.981 $\pm$ 0.005	5
TransE (reimplemented)	transductive	0.658 $\pm$ 0.024	0.424 $\pm$ 0.049	0.983 $\pm$ 0.005	5
Path-MP (2-hop)	transductive	0.988 $\pm$ 0.004	0.985 $\pm$ 0.003	0.990 $\pm$ 0.005	5
Rule oracle (hand-written)	inductive	0.950 $\pm$ 0.020	0.946 $\pm$ 0.019	0.952 $\pm$ 0.020	5
TransE+fold-in (reimplemented)	inductive	0.644 $\pm$ 0.025	0.423 $\pm$ 0.046	0.982 $\pm$ 0.010	5
TransE (reimplemented)	inductive	0.011 $\pm$ 0.002	0.000 $\pm$ 0.000	0.015 $\pm$ 0.006	5
Path-MP (2-hop)	inductive	0.989 $\pm$ 0.007	0.987 $\pm$ 0.008	0.992 $\pm$ 0.007	5

TABLE II
Filtered MRR per relation (mean $\pm$ std over seeds).

Method	Task	mrr_parent $\uparrow$	mrr_sibling $\uparrow$	mrr_grandparent $\uparrow$	mrr_uncle $\uparrow$	n
Rule oracle (hand-written)	transductive	0.828 $\pm$ 0.035	0.976 $\pm$ 0.020	0.996 $\pm$ 0.009	0.958 $\pm$ 0.036	5
TransE+fold-in (reimplemented)	transductive	0.735 $\pm$ 0.035	0.485 $\pm$ 0.010	0.858 $\pm$ 0.065	0.498 $\pm$ 0.052	5
TransE (reimplemented)	transductive	0.732 $\pm$ 0.045	0.485 $\pm$ 0.008	0.867 $\pm$ 0.056	0.499 $\pm$ 0.041	5
Path-MP (2-hop)	transductive	0.964 $\pm$ 0.019	0.999 $\pm$ 0.002	0.996 $\pm$ 0.009	0.994 $\pm$ 0.007	5
Rule oracle (hand-written)	inductive	0.851 $\pm$ 0.065	0.990 $\pm$ 0.002	1.000 $\pm$ 0.000	0.967 $\pm$ 0.033	5
TransE+fold-in (reimplemented)	inductive	0.692 $\pm$ 0.035	0.480 $\pm$ 0.006	0.863 $\pm$ 0.048	0.506 $\pm$ 0.033	5
TransE (reimplemented)	inductive	0.012 $\pm$ 0.004	0.010 $\pm$ 0.005	0.011 $\pm$ 0.003	0.012 $\pm$ 0.004	5
Path-MP (2-hop)	inductive	0.960 $\pm$ 0.025	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.998 $\pm$ 0.003	5

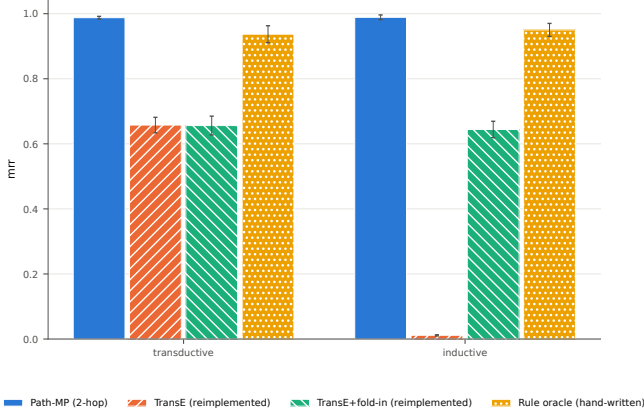


Fig. 1. Filtered MRR per system, in the two settings (mean and spread over seeds, as drawn by rh fig bars).

such as reaching a grandparent through a parent’s uncle edge.

Per relation (Table ??) and a failure case. TransE is best on grandparent (MRR 0.867) and worst on sibling and uncle (0.485 and 0.499). A plausible explanation, which we did not isolate with a run, is that sibling is symmetric and a translation model can represent a symmetric relation only with a near-zero relation vector, which makes it hard to separate siblings from other near neighbours; grandparent is a composition of parent, which translations represent naturally. Path-MP is nearly perfect on sibling, grandparent and uncle but weakest on parent (0.964

transductive, 0.960 inductive). The oracle is weakest on parent as well (0.828 transductive), consistent with a parent triple being recoverable only when a sibling with the same parent is observed, which is not the case for every held-out parent triple; Path-MP’s parent MRR is the clearest remaining failure of the model in this world.

H2: TransE collapses on new entities, Path-MP does not (supported). Frozen TransE gets MRR 0.011 on the new graph and Hits@1 0.000, a change of -0.646 relative to the transductive setting (paired $p = 4.28 \times 10^{-7}$). This is the expected result, since the vectors of the new entities are random; it is included as the reference point for the fold-in baseline. Path-MP’s change from transductive to inductive is 0.001 MRR (paired $p = 0.79$): within the 0.05 tolerance stated in H2, and statistically indistinguishable from zero with five seeds. The new graph is generated by the same process as the training graph, so this tests transfer to new entities, not to a different distribution of graphs.

H3: fold-in recovers most of TransE’s loss but stays below Path-MP (supported). TransE with fold-in reaches 0.644 on the new graph, 0.633 above frozen TransE (paired $p = 5.98 \times 10^{-7}$), and its inductive MRR is not distinguishable from TransE’s transductive MRR (change -0.012 , $p = 0.59$). Path-MP stays above fold-in by 0.345 MRR (paired $p = 1.24 \times 10^{-5}$). The conclusion that TransE cannot handle new entities is thus an artefact of the evaluation set-up: with fold-in, the inductive handicap of TransE vanishes in this world, and what remains is its gap to the path model in both settings. Fold-in costs

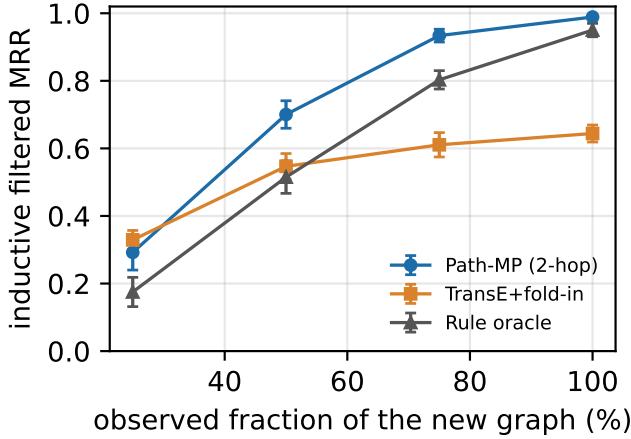


Fig. 2. Inductive MRR against the observed fraction of the new graph (mean $\pm$ std over seeds), from the same runs as Table IV.

training on the new graph at test time, while Path-MP only needs a forward pass.

VI. Ablations and Analysis

H4: depth and edge removal (supported). With one layer the model scores at chance (MRR 0.004 transductive, 0.004 inductive; a constant scorer under the even tie-splitting rule gets about $2/n$ with n the number of entities), a paired difference of -0.985 in the inductive setting. This is expected: with the query edge removed, no answer is within one hop of the query entity for any of the four relations, so the one-layer scores carry no information about the candidates; we did not run a probe that isolates this, so it is the likely, not a demonstrated, reason. A third layer changes nothing measurable: the paired difference to two layers is 0.002 (paired $p = 0.50$) transductively and -0.001 ($p = 0.73$) inductively; with five seeds this means “no difference detected”, not equivalence, and the model is at the ceiling so a deeper model has little room to help. Training without removing the query edge is the most damaging change after removing a layer: MRR falls to 0.507 (transductive) and 0.458 (inductive), differences of -0.480 and -0.531 to the main model (paired p below 0.001). With the edge present at training time the model can score the answer from the edge itself; at test time that edge is absent by construction. This is the likely mechanism; we did not measure which input the model relies on. The variance across seeds is high for this variant (std 0.090 inductive). In the no-removal variant sibling is hurt most (MRR 0.277 inductive), whereas uncle is hurt least (0.628); we have no isolating run for this ordering.

H5: sparser new graphs (supported for Path-MP; the fold-in baseline degrades too). Table IV and Fig. 2 vary the observed fraction of the new graph. Path-MP’s inductive MRR falls monotonically at the four grid points, from 0.989 with everything observed to 0.934, 0.700 and 0.292

at the three sparser levels; the step from the full to the 75% graph is itself significant (paired difference 0.055, $p = 3.76 \times 10^{-3}$). The rule oracle falls similarly, so the loss is largely a loss of information about the rules’ bodies in the observed graph rather than a defect of the model: a path model cannot answer a query whose supporting facts are not observed. Fold-in TransE degrades more gently (0.644 to 0.329) and the ranking of the systems changes with density. Path-MP is ahead of fold-in at the 75% and 50% levels (paired differences 0.323 and 0.153), but at 25% fold-in TransE has the higher mean (difference -0.037 for Path-MP, paired $p = 0.15$), which with five seeds is inconclusive. Path-MP stays above the oracle at all three sparser levels (paired p below 0.001). Note that in this sweep Path-MP is applied without retraining on sparse graphs; training on sparser graphs might narrow the gap, which we did not test.

VII. Limitations

- Synthetic, easy, high ceiling. Four relations are derived exactly by two-hop rules from facts that are present with high probability in the observed graph, and the rules are redundant. The path model is near the ceiling, so the study cannot rank path models, say anything about longer rules, noisy or non-rule facts, or realistic benchmarks, and the depth ablation cannot show an advantage of deeper models.
- Same-distribution new graph. The “fresh” graph is generated by the same process with the same relation vocabulary; the inductive setting here is new entities only, not new relations and not a distribution shift in graph structure or size.
- Scale and statistics. 5 seeds, graphs of 500 people, a few hundred test triples per seed; paired tests with five seeds detect only large effects, and we never interpret non-significance as equivalence (for example the third layer, or Path-MP versus fold-in TransE at 25% observed).
- Reimplemented, reduced baselines. TransE is our own implementation with a custom, modest negative-sampling and normalisation scheme; other tuned embedding models (RotatE, ComplEx) can represent symmetric and compositional patterns and would probably do better than TransE, so the gap between embeddings and Path-MP is an upper bound for the embedding family. We did not run GraIL, NBFNet, DRUM or ULTRA; Path-MP is a reduced NBFNet-style model. Tuning effort was larger for TransE than for Path-MP.
- Fold-in uses the observed new graph. The fold-in baseline needs gradient steps on the new graph; we did not tune its number of steps separately, and we did not compare against a cheaper inductive embedding scheme.

TABLE III

Path-MP ablations: MRR (mean $\pm$ std over seeds) and paired difference to the 2-layer model of Table I, in both settings. “No edge removal” trains with the query edge present in the message graph.

Variant	MRR trans.	MRR induct.	Δ vs L=2 (trans.)	Δ vs L=2 (induct.)
1 layer	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	-0.984	-0.985
2 layers (main)	0.988 $\pm$ 0.004	0.989 $\pm$ 0.007	—	—
3 layers	0.989 $\pm$ 0.004	0.988 $\pm$ 0.011	0.002	-0.001
2 layers, no edge removal	0.507 $\pm$ 0.073	0.458 $\pm$ 0.090	-0.480	-0.531

TABLE IV

Inductive MRR (mean $\pm$ std over seeds) when only a fraction of the new graph’s train+validation triples forms the message graph (or fold-in data); the test triples are unchanged. The 100% column is the main result.

System	25%	50%	75%	100%
Path-MP (2-hop)	0.292 $\pm$ 0.052	0.700 $\pm$ 0.041	0.934 $\pm$ 0.019	0.989 $\pm$ 0.007
TransE+fold-in	0.329 $\pm$ 0.028	0.547 $\pm$ 0.038	0.611 $\pm$ 0.036	0.644 $\pm$ 0.025
Rule oracle	0.175 $\pm$ 0.043	0.515 $\pm$ 0.048	0.803 $\pm$ 0.027	0.950 $\pm$ 0.020

- Oracle is not a ceiling. The rule oracle applies each rule once; the learned model exceeds it, so no result here should be read as “fraction of the achievable”.
- Mechanisms are hypotheses. Explanations for the one-layer failure, the no-edge-removal failure and TransE’s weakness on symmetric relations are plausible readings of the numbers, not isolated by dedicated runs. The advisory verdict tool of the pipeline compares the method with the rule oracle only and is not informative here.
- Split design. Units are held out at random; triples that cannot be inferred from the observed graph exist and bound the achievable MRR (the parent relation), but we did not count them.

VIII. Conclusion

On rule-derived family trees, a 2-layer relational message-passing model reached filtered MRR 0.988 on held-out triples and 0.989 on a fresh graph with new entities, with no measurable transfer loss, while a tuned TransE reached 0.658 on held-out triples and was at chance on new entities. The inductive failure of TransE is largely a consequence of the evaluation set-up: fitting only the new entity vectors on the observed new graph restores it to 0.644, but it remains far below the path model. The path model’s advantage is not unconditional: it needs two layers and training with the query edge removed, and when only a quarter of the new graph is observed it is not distinguishable from fold-in TransE. These conclusions hold for a small synthetic world with one generative process and five seeds; testing them on benchmarks with unknown rules, with tuned RotatE/ComplEx baselines and with distribution shift between graphs is the natural next step.

References

- [1] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” pp. 2787–2795, 2013.
- [2] B. Yang, W. tau Yih, X. He, J. Gao, and L. Deng, “Embedding Entities and Relations for Learning and Inference in Knowledge Bases,” arXiv preprint arXiv:1412.6575, 2014. [Online]. Available: <https://arxiv.org/abs/1412.6575>
- [3] T. Trouillon, J. Welbl, S. Riedel, Éric Gaussier, and G. Bouchard, “Complex Embeddings for Simple Link Prediction,” arXiv preprint arXiv:1606.06357, 2016. [Online]. Available: <https://arxiv.org/abs/1606.06357>
- [4] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, “RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space,” arXiv preprint arXiv:1902.10197, 2019. [Online]. Available: <https://arxiv.org/abs/1902.10197>
- [5] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, “Convolutional 2D Knowledge Graph Embeddings,” arXiv preprint arXiv:1707.01476, 2017. [Online]. Available: <https://arxiv.org/abs/1707.01476>
- [6] A. Rossi, D. Firmani, A. Matinata, P. Merialdo, and D. Barbosa, “Knowledge Graph Embedding for Link Prediction: A Comparative Analysis,” arXiv preprint arXiv:2002.00819, 2020. [Online]. Available: <https://arxiv.org/abs/2002.00819>
- [7] K. K. Teru, E. Denis, and W. L. Hamilton, “Inductive Relation Prediction by Subgraph Reasoning,” arXiv preprint arXiv:1911.06962, 2019. [Online]. Available: <https://arxiv.org/abs/1911.06962>
- [8] Z. Zhu, Z. Zhang, L.-P. Xhonneux, and J. Tang, “Neural Bellman-Ford Networks: A General Graph Neural Network Framework for Link Prediction,” arXiv preprint arXiv:2106.06935, 2021. [Online]. Available: <https://arxiv.org/abs/2106.06935>
- [9] H. Wang, H. Ren, and J. Leskovec, “Relational Message Passing for Knowledge Graph Completion,” arXiv preprint arXiv:2002.06757, 2020. [Online]. Available: <https://arxiv.org/abs/2002.06757>
- [10] Y. Geng, J. Chen, J. Z. Pan, M. Chen, S. Jiang, W. Zhang, and H. Chen, “Relational Message Passing for Fully Inductive Knowledge Graph Completion,” arXiv preprint arXiv:2210.03994, 2022. [Online]. Available: <https://arxiv.org/abs/2210.03994>
- [11] F. Yang, Z. Yang, and W. W. Cohen, “Differentiable Learning of Logical Rules for Knowledge Base Reasoning,” arXiv preprint arXiv:1702.08367, 2017. [Online]. Available: <https://arxiv.org/abs/1702.08367>
- [12] A. Sadeghian, M. Armandpour, P. Ding, and D. Z. Wang, “DRUM: End-To-End Differentiable Rule Mining On Knowledge Graphs,” arXiv preprint arXiv:1911.00055, 2019. [Online]. Available: <https://arxiv.org/abs/1911.00055>
- [13] M. Galkin, M. Berrendorf, and C. T. Hoyt, “An Open Challenge for Inductive Link Prediction on Knowledge Graphs,” arXiv preprint arXiv:2203.01520, 2022. [Online]. Available: <https://arxiv.org/abs/2203.01520>
- [14] M. Galkin, X. Yuan, H. Mostafa, J. Tang, and Z. Zhu, “Towards Foundation Models for Knowledge Graph Reasoning,” arXiv preprint arXiv:2310.04562, 2023. [Online]. Available: <https://arxiv.org/abs/2310.04562>
- [15] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, S. Sharifzadeh, V. Tresp, and J. Lehmann, “PyKEEN 1.0: A Python Library

for Training and Evaluating Knowledge Graph Embeddings,”
arXiv preprint arXiv:2007.14175, 2020. [Online]. Available:
<https://arxiv.org/abs/2007.14175>

- [16] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov,
and M. Welling, “Modeling Relational Data with Graph
Convolutional Networks,” arXiv preprint arXiv:1703.06103,
2017. [Online]. Available: <https://arxiv.org/abs/1703.06103>